



UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E
TELECOMUNICAZIONE
CORSO DI LAUREA MAGISTRALE IN
INFORMATICA

Relazione di Progetto Compressione delle immagini

Studenti:

Acquati Luca
Matricola 934996

Jamil Muhammad Qasim
Matricola 932972

Anno accademico:

2024/2025

Indice

Capitolo 1. Introduzione	1
1.1 Immagini JPEG	1
Capitolo 2. Progetto	3
2.1 Struttura del Progetto	3
2.2 Gestione delle immagini	3
2.3 Calcolo della DCT	3
2.3.1 MSE	4
2.3.2 PSNR	4
2.4 Compressione dell'immagine	4
2.5 Interfaccia grafica	5
Capitolo 3. Risultati ottenuti	7
3.1 Valori bassi di F e d	7
3.2 Valori elevati di F e d	8
Capitolo 4. Conclusioni	11

Capitolo 1

Introduzione

L'obiettivo di questa seconda parte è di sviluppare un'applicazione che permetta all'utente di selezionare un'immagine, configurare parametri di compressione e visualizzare gli effetti della compressione in termini di perdita di dettaglio e qualità visiva.

Viene implementata una tecnica di compressione ispirata al principio del JPEG, ma semplificata: la riduzione delle informazioni avviene tramite la soppressione selettiva delle componenti ad alta frequenza nei blocchi locali dell'immagine, senza l'utilizzo di matrici di quantizzazione standard.

L'utente ha la possibilità di controllare la dimensione dei blocchi e la soglia di eliminazione delle frequenze, con un'interfaccia semplice e intuitiva.

1.1 Immagini JPEG

Le immagini possono essere viste come una sequenza di $N \times M$ pixel e il formato jpeg combacia con l'applicazione della DCT2, si comprime l'immagine e per ritornare all'immagine iniziale, si applica IDCT2. Il problema è che si ottiene una matrice con la virgola mentre i pixel sono interi, perciò è necessario arrotonda la matrice e normalizzare le entry nell'intervallo $[0,255]$

Capitolo 2

Progetto

Il progetto è stato creato utilizzando Python ed organizzato in modo da separare chiaramente le funzionalità, favorendo la leggibilità e la facilità di manutenzione.

2.1 Struttura del Progetto

Il progetto è suddiviso nelle seguenti directory:

- **utils**: cartella contenente i file per la gestione delle immagini, l'esecuzione della DCT e la gestione dei percorsi;
- **images**: cartella con le immagini su cui eseguire la compressione;
- **compressed_images**: cartella in cui verranno salvate le immagini compresse.

Nella cartella di progetto sono presenti i seguenti file:

- **gui.py**: file per la gestione dell'interfaccia grafica;
- **main.py**: file principale per l'esecuzione del progetto.

2.2 Gestione delle immagini

La classe `ImageUtils` fornisce una serie di funzioni per la gestione delle immagini in ambiente Tkinter e per la creazione automatica di confronti visivi tra l'immagine originale e quella compressa. La funzione `display_image` consente di visualizzare correttamente un'immagine all'interno di un canvas Tkinter. La funzione `save_comparison` permette di generare e salvare automaticamente un'immagine di confronto tra l'originale e la versione compressa, l'output prodotto è un'immagine affiancata che mostra le due versioni, utile per una valutazione visiva e immediata dei risultati ottenuti. Il confronto viene salvato nella directory `compressed_images`.

2.3 Calcolo della DCT

La gestione della DCT2 è stata implementata nel modulo `dct_utils.py`, che contiene la classe `DCTProcessor`. Questo componente si occupa di applicare

la DCT bidimensionale a blocchi di un'immagine in scala di grigi e di gestire la successiva ricostruzione dell'immagine compressa.

La classe `DCTProcessor` consente di applicare la DCT2 e la sua inversa utilizzando le funzioni `dctn` e `idctn` fornite dalla libreria SciPy. Inoltre, la classe permette di comprimere singoli blocchi di immagine eliminando selettivamente le componenti ad alta frequenza in base a una soglia definita dall'utente. La stessa logica viene estesa all'intera immagine, che viene suddivisa in macro-blocchi di dimensione variabile, ciascuno dei quali viene processato in modo indipendente per ottenere la versione compressa dell'immagine originale.

Vengono anche calcolate diverse statistiche utili per valutare l'efficacia della compressione, tra cui l'errore quadratico medio (MSE), il rapporto segnale-rumore di picco (PSNR) e il rapporto di compressione, fornendo così una misura oggettiva della perdita di informazione introdotta dal processo di compressione.

2.1: Calcolo della DCT2

```
def dct2_library(self, block):  
    return dctn(block, type=2, norm='ortho')
```

2.3.1 MSE

Il Mean Squared Error (MSE) è una misura quantitativa della qualità della ricostruzione di un'immagine rispetto all'originale. Viene calcolato come la media dei quadrati delle differenze tra i pixel corrispondenti delle due immagini. Valori più bassi di MSE indicano una maggiore somiglianza tra l'immagine compressa e quella originale, mentre valori più alti suggeriscono una maggiore perdita di informazione.

2.3.2 PSNR

Il PSNR valuta quanto rumore viene introdotto tra un'immagine originale e una modificata (compressa, ricostruita). È espresso in decibel (dB), più è alto, migliore è la qualità cioè vi è meno differenza tra immagine originale e immagine modificata.

2.4 Compressione dell'immagine

Il metodo `compress_image` della classe `DCTProcessor` suddivide l'immagine in blocchi quadrati di dimensione $F \times F$, comprime ciascun blocco in modo indipendente e ricompone l'immagine compressa scartando eventuali pixel di bordo non inclusi nei blocchi completi.

```

def compress_image(self, image, F, d, progress_callback=None):
    height, width = image.shape

    blocks_h = height // F
    blocks_w = width // F

    # Creo l'immagine compressa
    compressed_height = blocks_h * F
    compressed_width = blocks_w * F
    compressed_image = np.zeros((compressed_height,
                                compressed_width), dtype=np.uint8)

    # Processo ogni blocco
    total_blocks = blocks_h * blocks_w

    for i in range(blocks_h):
        for j in range(blocks_w):
            start_row = i * F
            end_row = start_row + F
            start_col = j * F
            end_col = start_col + F

            block = image[start_row:end_row, start_col:end_col]

            compressed_block = self.compress_block(block, d)

            compressed_image[start_row:end_row, start_col:end_col]
                = compressed_block

            # Aggiorna progresso
            if progress_callback:
                progress = ((i * blocks_w + j + 1) / total_blocks)
                    * 100
                progress_callback(progress)

    # Calcola statistiche
    stats = self.calculate_stats(image[:compressed_height, :
                                        compressed_width], compressed_image, F, d)

    return compressed_image, stats

```

2.5 Interfaccia grafica

L'applicazione include una semplice interfaccia grafica che consente all'utente di selezionare un'immagine, impostare i parametri di compressione e visualizzare il risultato ottenuto. La GUI è stata realizzata utilizzando la libreria open source Tkinter.

La finestra principale permette all'utente di selezionare un'immagine BMP in scala di grigi tramite selettore di file, impostare la dimensione F dei blocchi su cui applicare la DCT2 e definire la soglia d per la soppressione dei coefficienti di frequenza. La GUI aggiorna dinamicamente il limite massimo selezionabile per il parametro d in base al valore scelto per F garantendo la coerenza dei parametri inseriti. Durante la compressione, viene mostrata una barra di progresso per indicare lo stato di avanzamento dell'elaborazione e un pannel-

lo testuale in cui vengono visualizzate informazioni di stato e le statistiche relative alla compressione, come l'errore quadratico medio (MSE), il PSNR e il rapporto di compressione.

Sono presenti anche due canvas distinti per visualizzare affiancate l'immagine originale e la versione compressa, consentendo una valutazione visiva degli effetti della compressione.

Al termine del processo, l'applicazione salva automaticamente un'immagine di confronto in una cartella dedicata.

Capitolo 3

Risultati ottenuti

I test di compressione hanno messo in evidenza un comportamento particolare derivante dai valori utilizzati per i parametri F e d .

3.1 Valori bassi di F e d

Con valori bassi sia di F che di d , si è osservato che le immagini molto dettagliate (ad esempio fotografie) mantengono quasi inalterato l'aspetto visivo, poiché i blocchi piccoli contengono poche informazioni e la soglia bassa non riesce a eliminare completamente le frequenze significative.

In questo modo l'effetto visivo su scala globale è minimo o invisibile come nell'immagine riportata di seguito:



Figura 3.1: Confronto con $d = 1$ e $F = 0$

Al contrario, in immagini poco dettagliate la stessa configurazione porta a un risultato quasi completamente nero o piatto, in quanto la perdita anche di poche frequenze è sufficiente a cancellare tutta l'informazione utile.

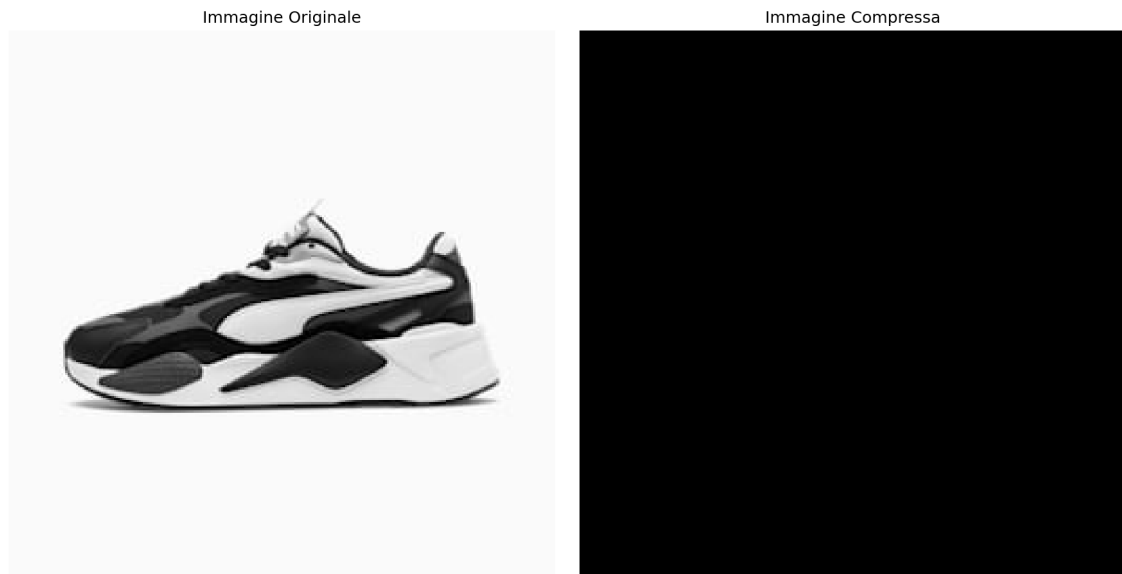


Figura 3.2: Confronto $d = 1$ e $F = 0$ con pochi dettagli

3.2 Valori elevati di F e d

Come per i valori bassi anche con valori elevati di F e d per immagini molto dettagliate viene mantenuto inalterato l'aspetto visivo.



Figura 3.3: Confronto immagine poco dettagliata con taglio dell'immagine $d = 30$ e $F = 16$

Con immagini poco dettagliate l'effetto si inverte: la compressione è minima, ma l'immagine ricostruita appare “tagliata”, poiché solo una parte dell'imma-

gine originale è effettivamente ricostruita, mentre i bordi o le aree periferiche restano vuote o troncate.

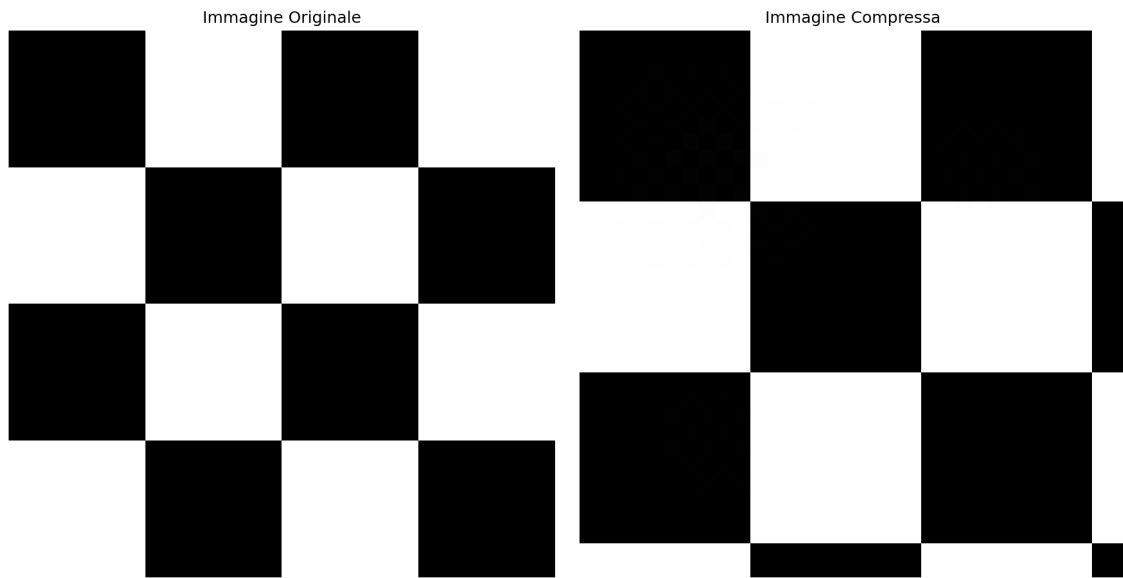


Figura 3.4: Confronto immagine poco dettagliata con taglio dell'immagine $d = 30$ e $F = 16$

Questo è dovuto al fatto che l'immagine non sempre è perfettamente divisibile per F e i blocchi incompleti vengono scartati, come previsto dalla specifica.

Capitolo 4

Conclusioni

L'applicazione ha dimostrato una buona efficienza nella gestione dei blocchi e nell'elaborazione tramite la DCT2 soprattutto con immagini poco dettagliate.

I test effettuati sulle immagini hanno confermato che la qualità percepita dell'immagine compressa dipende in modo significativo dalla scelta della dimensione dei blocchi F dalla soglia d e dall'immagine scelta.

Per immagini poco dettagliate valori troppo bassi di d portano a un'eccessiva perdita di informazione, mentre valori elevati preservano maggiormente i dettagli, ma con minore compressione e tagli dell'immagine.

Per immagini molto dettagliate, come ad esempio fotografie, sia i valori bassi che alti di d e F portano a modifiche praticamente impercettibili mantenendo l'immagine identica.